

Atividade: paralelismo na camada de aplicação | Justificativa

Por que escolhemos a US07

1. A operação escolhida

US07 | Análise Preditiva de Demanda por Tipo Sanguíneo

História: como analista de dados do sistema, eu gostaria de visualizar as projeções de demanda futura e os desvios estatísticos de consumo, para que as campanhas de doação sejam direcionadas aos tipos sanguíneos com maior risco de desabastecimento.

Dado que a base histórica de consumo dos últimos meses foi processada

Quando o analista acessa o módulo de "Demanda Preditiva" para o próximo trimestre

Então o sistema plota a média estimada (μ) e as faixas de dispersão (σ) para cada tipo sanguíneo (A, B, AB, O, Rh+/-)

E destaca os tipos com probabilidade de ruptura de estoque (ex.: O-).

Implementamos a US07 como o endpoint `GET /api/demanda/preditiva` (Java 21 + Spring Boot). A cada chamada ele:

1. **percorre todo o histórico** de requisições das últimas 104 semanas; para cada uma, descarta as canceladas, converte o instante gravado em UTC para a **data local do hospital** (o Brasil tem 4 fusos) e encontra a **semana epidemiológica**;
2. **soma as bolsas** na célula tipo sanguíneo $\times$ semana: 8 tipos $\times$ 104 semanas = 832 células;
3. calcula, por tipo, **μ , σ , tendência** (regressão linear) e a **projeção das próximas 13 semanas**;
4. compara a projeção com o estoque e calcula **P(ruptura)**, destacando os tipos em alerta (O- em risco alto).

2. Como chegamos a ela

Seguimos o roteiro da atividade: para cada operação perguntamos onde o tempo é gasto na escala nacional, qual a Big-O e se os dados se dividem em fatias independentes.

História	Onde o tempo é gasto	Big-O	Particionável?	Veredito
US01 Requisição	banco (1 INSERT)	$O(1)$	não há volume	não serve
US02 Entrada de bolsa	banco (1 INSERT)	$O(1)$	não há volume	não serve
US03 Alocação FEFO	consulta com índice	$O(\log n)$	não: disputa pelo estoque	não serve
US04 Rota (Dijkstra)	CPU, grafo pequeno	$O((V+E) \log V)$	só em lote de rotas	candidata fraca
US05 Telemetria	rede, eventos simultâneos	$O(1)$ por leitura	concorrência, não volume	não serve
US06 Painel gerencial	contagens e médias	$O(n)$ no banco	sim	melhorar o SQL resolve
US07 Demanda preditiva	CPU, milhões de registros	$O(n)$	sim	escolhida

3. Big-O da solução sequencial

Sejam **n** o número de requisições no histórico, **t = 8** tipos sanguíneos e **w = 104** semanas.

etapas 1 e 2: visitar cada registro uma vez **$O(n)$**
etapas 3 e 4: percorrer 8 séries de 104 semanas ... $O(t \cdot w) = O(832)$, constante
total: $O(n + t \cdot w) = O(n)$ memória extra: $O(t \cdot w)$

A medição confirma a teoria: o custo fica em torno de **49 ns por registro** de 1 a 10 milhões de registros (≈ 49 ms para 1 milhão e ≈ 494 ms para 10 milhões). Dobrar a base dobra o tempo: crescimento linear.

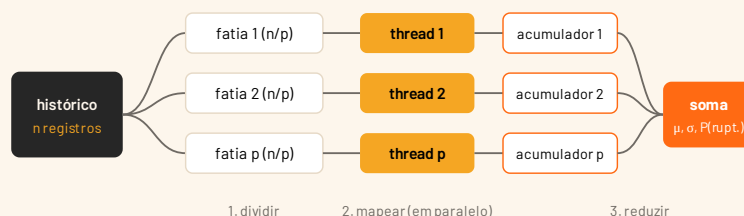
4. Onde está o gargalo

Na escala nacional são milhares de hospitais e dezenas de milhões de requisições no histórico, e o painel preditivo é consultado o dia inteiro, com o analista mudando o período e os parâmetros. Por isso a base histórica é lida do banco **uma vez** para um **snapshot em memória** na camada de aplicação (atualizado periodicamente), e cada chamada ao endpoint trabalha sobre esse snapshot. O gargalo, então, é **processamento**, não espera:

- **O tempo do endpoint não tem espera de banco nem de rede interna.** É 100% laço sobre os registros: com 10 milhões são ~0,5 s de CPU em uma única thread, a cada consulta.
- **O trabalho por registro é cálculo:** conversão do instante UTC para a data local com as regras de fuso de cada hospital, cálculo da semana epidemiológica e acumulação.
- **Não é caso de "melhorar o SQL".** Mesmo com a melhor consulta, depois que os dados estão na aplicação continua sobrando $O(n)$ de cálculo, e o banco transacional já está ocupado com o caminho crítico: requisições emergenciais (US01), entradas de bolsas (US02) e telemetria (US05). Rodar varreduras analíticas nele o dia todo competiria com essas escritas; a estatística e o modelo de projeção vivem na aplicação.

5. Por que os dados são particionáveis

- **Registros independentes:** cada requisição contribui para uma única célula tipo $\times$ semana e não depende de nenhuma outra.
- **Agregação associativa e comutativa:** somar inteiros (`long`) em qualquer ordem dá o mesmo total. Dá para somar p fatias separadas e depois somar os p parciais.
- **Leitura compartilhada, escrita local:** a lista é somente leitura; cada thread escreve só no seu acumulador `Long[8][104]`. Sem escrita compartilhada no laço, não há `race condition` nem trava.
- **Redução barata:** juntar os parciais custa $p \times 832$ somas, desprezível perto de n .



trabalho total: $O(n + p \cdot t \cdot w) = O(n)$
tempo com p núcleos: $O(n/p + p \cdot t \cdot w)$

A Big-O do **trabalho** não muda (cada registro continua visitado uma vez); o que muda é que ele é dividido entre p núcleos. Implementamos a versão paralela com `ExecutorService` (2, 4 e 8 threads), cada thread processando a própria fatia, com agregação ao final, e também com as threads virtuais do Java 21.

Como provamos que a resposta é exatamente a mesma: todo resultado traz uma assinatura SHA-256 de todos os números agregados. Os testes automatizados e o endpoint `/api/demanda/verificar` confirmam a mesma assinatura (`1ff7dd2efde0e8aa` para 1 milhão de registros) em todas as versões. Como contraprova, uma versão errada de propósito, em que as 8 threads somam no mesmo acumulador sem sincronização, perdeu entre 2.366 e 2.867 bolsas em 5 de 5 rodadas.

Em resumo

A US07 é a única operação do HemoTrack que, na escala nacional, gasta o tempo calculando sobre milhões de registros ($O(n)$) e cujos dados se dividem em fatias independentes com soma associativa. Por isso o paralelismo com threads traz ganho mensurável sem mudar a resposta: na máquina do grupo (16 núcleos), 1 milhão de registros caiu de 90,1 ms para 33,6 ms com 4 threads (speedup de 2,68x).